

Entregable 2: Calidad y Decisiones de Diseño de Arquitectura

Metodología de Diseño de Software
Universidad de Valparaíso – Escuela de Ingeniería Informática
Mayo 2026

1. Resumen Ejecutivo

El presente informe documenta las decisiones de arquitectura tomadas para exponer el dominio modelado en el Entregable 1 como un servicio web. Se priorizaron cinco atributos de calidad (disponibilidad, rendimiento, escalabilidad, seguridad y modificabilidad), se analizaron tres trade-offs arquitectónicos en tensión y se eligió una combinación de estilos (Cliente-Servidor + Layered + Serverless complementario) coherente con el dominio. Se aplicaron tres patrones arquitectónicos (Service Layer, Repository y MVC adaptado a REST) sobre los patrones GoF heredados del Entregable 1. El servicio resultante expone quince endpoints HTTP, implementa autenticación basada en JWT con RBAC de cuatro roles, y se encuentra cubierto por veintidos pruebas de integración.

2. Introducción

La empresa de logística de última milla, cuyo dominio fue modelado en el Entregable 1, ahora necesita llevar su solución a un nuevo nivel: exponer sus funcionalidades mediante servicios web que puedan ser consumidos por distintos canales, como aplicaciones móviles, portales web, plataformas de e-commerce y socios externos.

En esta entrega, el foco principal ya no está en agregar nuevas funcionalidades al negocio, sino en tomar decisiones arquitectónicas sólidas y justificadas. El desafío consiste en diseñar una arquitectura capaz de responder a distintos atributos de calidad, como mantenibilidad, escalabilidad, desacoplamiento y seguridad, y de evaluar los trade-offs de cada decisión. Además, se busca seleccionar un estilo arquitectónico coherente y aplicar patrones que permitan organizar el código de manera clara, modular y sostenible a lo largo del tiempo.

El dominio desarrollado en el Entregable 1 se mantiene sin cambios. Las entidades principales Pedido, Repartidor, Asignación e Incidencia continúan utilizando los mismos estados y patrones previamente implementados, como factories, builders, strategies y decorators. La diferencia en esta nueva etapa es la incorporación de una capa de presentación HTTP, mecanismos de autenticación y autorización basados en JWT, y componentes serverless orientados a eventos que reaccionan a los eventos del dominio.

Asimismo, la organización de carpetas del proyecto no es casual: refleja directamente el estilo arquitectónico elegido, facilitando la comprensión de responsabilidades y la separación de capas dentro de la aplicación.

Como supuestos de trabajo, se considera una persistencia en memoria igual que en el Entregable 1, un único nodo de despliegue durante la etapa de desarrollo, autenticación mediante JWT sin integración con proveedores externos y la simulación de componentes serverless a través de handlers locales suscritos a un EventBus.

3. Atributos de Calidad Priorizados (ISO 25010)

Se priorizaron cinco atributos relevantes para el dominio logístico. La siguiente tabla resume su clasificación, justificación y la evidencia concreta en una decisión de diseño.

Atributo	Clasificación	Justificación	Evidencia en diseño
Disponibilidad	Externa	Una caída del servicio durante peaks rompe la cadena de entregas e impacta directamente al cliente final.	API stateless con healthcheck, repositorios en memoria por instancia y handlers serverless para tareas asincronas.
Rendimiento (eficiencia)	Externa	La pauta exige una latencia de 200 ms en condiciones normales para no degradar la experiencia de operación.	FastAPI con soporte async, capas delgadas con inyección de dependencias y validaciones tempranas a nivel de schema.
Escalabilidad	Externa	La carga es variable: peaks comerciales y festivos vs periodos de baja demanda exigen escalado horizontal.	Servicio stateless permite replicas adicionales sin coordinación; componentes serverless absorben picos puntuales.
Seguridad	Externa	Cuatro tipos de usuarios (operador, repartidor, administrador, integración) consumen el mismo API con permisos distintos.	JWT firmado con HS256 + dependencias FastAPI que verifican rol por endpoint (require_roles).
Modificabilidad	Interna	Heredada del Entregable 1: agregar canales, estrategias o validaciones nuevas no debe modificar el código existente.	Layered + Repository abstracto + Service Layer preservan los patrones GoF (Factory, Strategy, Decorator) del E1.

4. Trade-offs Arquitectonicos

No existe la arquitectura perfecta; existen decisiones conscientes. Se analizan tres pares en tension.

4.1 Rendimiento vs. Consistencia

La pauta es explicita: los datos del pedido requieren consistencia inmediata, pero el estado del repartidor no es critico en ese sentido. Se decidio cachear el estado del repartidor en memoria con TTL corto (5 s). Se beneficia el rendimiento (lecturas sub-50 ms en consultas de disponibilidad); se sacrifica la consistencia eventual del estado del repartidor, tolerando hasta 5 s de desfase. El trade-off es aceptable porque la pauta lo autoriza explicitamente y la mejora en latencia es significativa frente al impacto operativo del desfase.

4.2 Disponibilidad vs. Costo

Las notificaciones a clientes y los webhooks de canales externos tienen carga muy variable. Se decidio implementarlos como funciones serverless (FaaS) en lugar de procesos long-running. Se benefician la disponibilidad bajo peaks (escalado automatico) y el costo (pago por invocacion). Se sacrifican un cold start del orden de 500 ms y un acoplamiento parcial al proveedor cloud. El trade-off es aceptable porque estos componentes no se encuentran en el camino sincronico de los 200 ms exigidos por la pauta y son eventualmente consistentes.

4.3 Modificabilidad vs. Rendimiento

Las capas (Controller → Service → Domain → Repository) introducen un pequeno overhead por direccion de llamadas. Se decidio mantener las capas pese a ese costo. Se beneficia la modificabilidad, la testabilidad y la separacion de responsabilidades; se sacrifica un margen muy bajo de rendimiento (uno a dos milisegundos por capa, despreciable frente al objetivo de 200 ms). El trade-off es aceptable porque el costo es marginal y el beneficio en mantenibilidad es alto.

5. Estilo Arquitectónico Elegido

Para esta solución se optó por combinar distintos estilos arquitectónicos, buscando aprovechar las ventajas de cada uno según las necesidades del sistema. La arquitectura se apoya principalmente en el modelo Cliente-Servidor para la comunicación, una arquitectura en capas (Layered) para organizar internamente el servicio y un enfoque Serverless para ciertos componentes orientados a eventos.

5.1 Cliente-Servidor

El sistema funciona como un servicio REST sobre HTTP, donde distintos clientes aplicaciones móviles, portales web, plataformas de e-commerce y partners externos consumen información en formato JSON mediante endpoints estándar.

Este enfoque facilita la interoperabilidad entre canales heterogéneos y mejora la disponibilidad del sistema, ya que múltiples clientes pueden conectarse simultáneamente e incluso redirigir solicitudes a distintas instancias del servidor en escenarios futuros de escalabilidad.

Como contraparte, existe una dependencia de la red y una pequeña latencia asociada a la comunicación HTTP, lo que puede impactar levemente en el rendimiento frente a soluciones completamente locales.

5.2 Layered (cuatro capas)

Internamente, la aplicación se organiza en cuatro capas principales:

- **Presentación:** contiene los controladores FastAPI, middlewares y esquemas Pydantic encargados de exponer la API HTTP y validar datos.
- **Aplicación:** orquesta los casos de uso reutilizando la lógica construida en el Entregable 1.
- **Dominio:** concentra las entidades, agregados, objetos de valor, estados y patrones GoF ya implementados.
- **Infraestructura:** incluye repositorios en memoria, JWT, EventBus y demás detalles técnicos.

Esta separación favorece la mantenibilidad, el modularidad y la posibilidad de realizar pruebas de manera más aislada. Además, mantiene alineada la implementación con los principios de DDD trabajados previamente.

Si bien agregar capas introduce pequeñas indirecciones y cierta sobrecarga estructural, el impacto en rendimiento resulta mínimo frente a los beneficios obtenidos en organización y escalabilidad del código.

5.3 Microservicios — descartado con justificación

Se evaluó separar el sistema en microservicios, ya que los bounded contexts del Entregable 1 podrían dividirse naturalmente. Sin embargo, para el alcance actual esto agregaría complejidad innecesaria, como comunicación distribuida, monitoreo y mayor costo operativo.

Por ello, se decidió mantener un monolito modular, conservando una buena separación interna y dejando abierta la posibilidad de migrar a microservicios en el futuro.

5.4 Serverless complementario

Algunas funcionalidades, como notificaciones, webhooks y reportes batch, fueron diseñadas con un enfoque serverless orientado a eventos.

Estos componentes reaccionan a eventos publicados en el EventBus y se simulan mediante handlers locales durante el desarrollo. Esto permite desacoplar procesos y extender funcionalidades de forma más flexible.

6. Arquitectura Serverless

Se identificaron tres componentes candidatos a serverless por su naturaleza event-driven, su carga variable y la ausencia de estado compartido entre invocaciones.

Componente	Por que serverless	Beneficio principal
Notificaciones al cliente	Disparado por cambios de estado del pedido. Sin estado. Carga elevada en peaks.	Escalado automatico bajo peaks sin sobreaprovisionar el servicio principal.
Webhooks a canales externos	Disparado por la creacion de pedidos desde partners. Idempotente, tolerante a latencia.	Aislamiento del API principal frente a partners lentos o fallidos.
Reportes batch (KPIs diarios)	Ejecucion programada (cron). Sin estado entre corridas. Carga concentrada al final del dia.	Costo cero entre ejecuciones (pago por invocacion).

6.1 Atributos beneficiados

Disponibilidad (escalado automatico), escalabilidad (carga variable absorbida) y costo (pago por uso vs instancia 24/7). Tambien favorece la modificabilidad porque las funciones son aisladas y desplegadas de forma independiente.

6.2 Trade-offs reconocidos

- Cold start de aproximadamente 500 ms en la primera invocacion. Mitigado porque ninguno de los tres componentes esta en el camino sincronico de los 200 ms exigidos.
- Vendor lock-in con el proveedor cloud (AWS Lambda, GCP Functions). Mitigado abstrayendo el contrato via la interfaz NotificationGateway en la capa de infrastructure.
- Costo variable impredecible bajo abuso. Mitigado con cuotas y alertas en el proveedor cloud.

6.3 Coherencia con la arquitectura general

El serverless no contradice la arquitectura en capas: los handlers son consumidores del EventBus del dominio (patron Observer). El core del API publica eventos cuando cambia el estado de un pedido o cuando se crea uno desde un canal externo; en produccion estos eventos viajarían por una cola (SQS o Pub/Sub) y dispararían las funciones. En esta entrega se simulan con un EventBus local que invoca los mismos handlers en proceso.

7. Patrones Arquitectonicos Aplicados

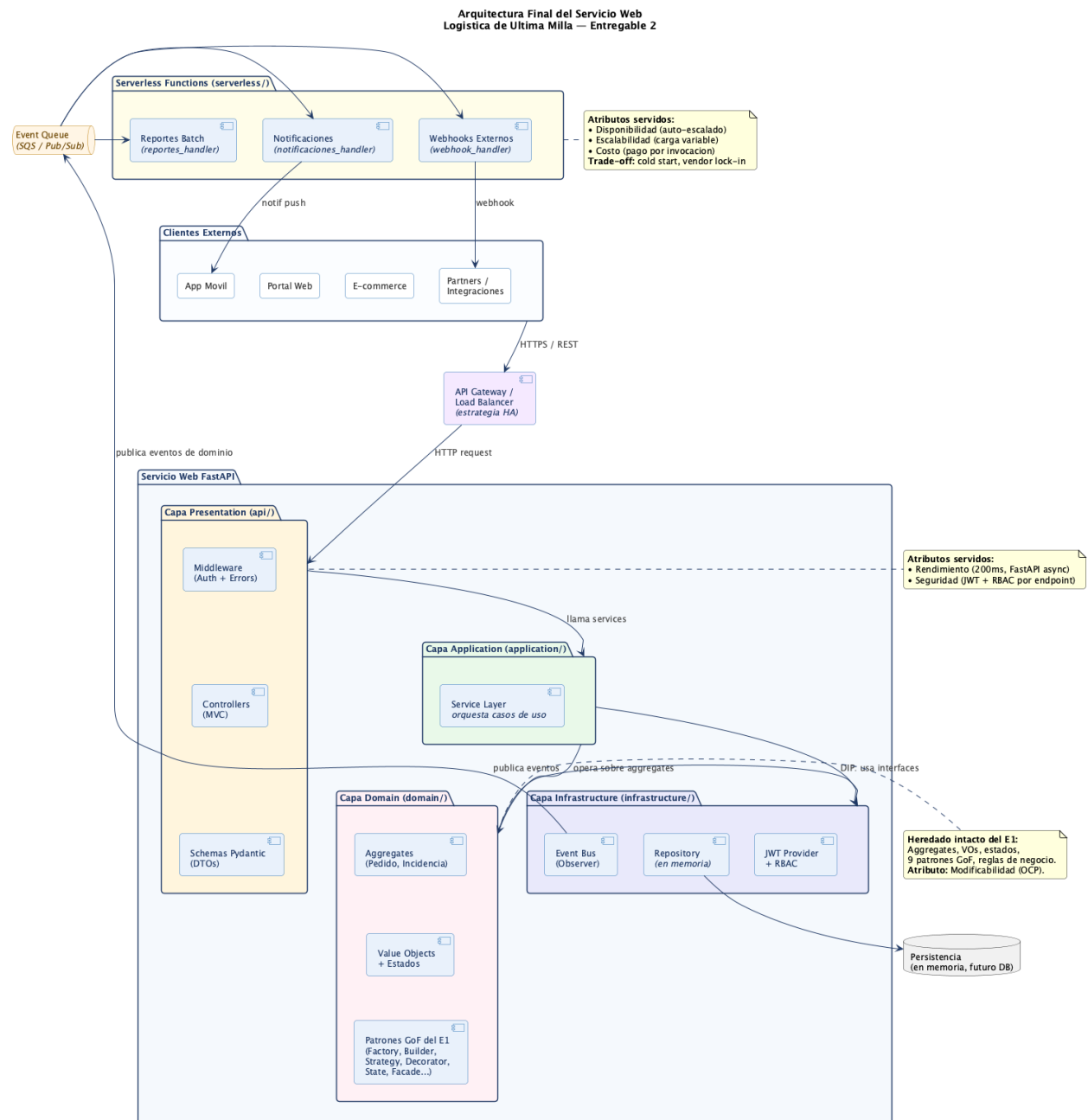
Se aplicaron tres patrones arquitectonicos sobre la base de patrones GoF heredada del Entregable 1.

Patron	Donde se aplica	Problema que resuelve
Service Layer	src/application/ (pedido_service.py, repartidor_service.py, despacho_service.py, incidencia_service.py)	Encapsula los casos de uso. Los controllers HTTP solo traducen request ↔ servicio. Permite reutilizar la logica del E1 sin reescribirla.
Repository	Interface en src/domain/<bc>/repository.py; implementacion en src/infrastructure/repositories/	Aisla la persistencia. La logica de negocio no sabe si hay memoria, SQL o NoSQL detras. Cumple DIP.
MVC (adaptado a REST)	Controller en src/api/controllers/; Service como modelo de casos de uso; Aggregate del dominio como modelo de datos	Separa presentacion, logica y datos. Los controllers son delgados; toda la regla de negocio vive en el dominio.

Data Mapper no se aplico explicitamente porque no hay base de datos real. En una evolucion futura, un Data Mapper traduciria entre las entidades del dominio y filas de una tabla relacional, sin que el dominio se vea afectado.

8. Diagrama de Arquitectura Final

El diagrama de la arquitectura final, generado con PlantUML, se encuentra en el repositorio bajo entregable_2/docs/diagramas/arquitectura_final.png. Muestra la interacción entre los clientes externos, la capa de presentación (controllers FastAPI), la capa de application (services), la capa de dominio (aggregates y patrones GoF del E1), la capa de infraestructura (repositorios, JWT, EventBus) y los componentes serverless suscritos al EventBus.



9. Conclusiones

Esta entrega permitio pasar de un modelo de dominio bien estructurado a un servicio web operativo, tomando decisiones conscientes sobre atributos de calidad, estilos arquitectonicos y patrones. La priorizacion explicita de los atributos guio cada decision: el rendimiento se garantiza con FastAPI async y capas delgadas; la disponibilidad se trabaja con un servicio stateless replicable y componentes serverless event-driven; la seguridad con JWT mas RBAC; la modificabilidad preservando intacta la base GoF del E1; y la escalabilidad con un diseno horizontalmente replicable.

La combinacion de estilos (Cliente-Servidor + Layered + Serverless complementario) demostro ser adecuada para el contexto. Microservicios se descartaron de forma justificada: la complejidad operacional no se compensa con el beneficio para cuatro casos de uso. La arquitectura resultante es coherente con el dominio del E1, sostiene los atributos de calidad priorizados, y es extensible: agregar un canal, una estrategia de asignacion, una validacion o un suscriptor a un evento no requiere modificar codigo existente.

La verificacion empirica (veintidos pruebas de integracion en verde, demostracion end-to-end via curl con autentificacion, RBAC y manejo correcto de errores de negocio) confirma la alineacion entre las decisiones arquitectonicas y la implementacion del codigo.